Decidable Type-Checking for the Dependent Categorical Combinators

Lexi Aiello[1] and Eric Klavins[2]

[1]Department of Philosophy, University of Washington
[2]Electrical & Computer Engineering Department, University of Washington

April 28, 2026

Abstract

We present an extension of the categorical combinators [4] to symmetric monoidal categories, enabling affine dependent types of the nonlinear categorical combinator terms. We formalize the metatheory in the Lean 4 proof assistant. We prove several important properties of the calculus, including that type-checking is decidable, and that the affine restriction of the combinators is strongly-normalizing, and globally confluent. We achieve judgmental decidability in principle by showing that the types of all objects can be expressed using the affine restriction, which we demonstrate is strongly-normalizing by a syntactic decreasing measure. We outline key challenges and architectural strategies for formalizing a dependent combinator calculus with such properties in a proof assistant, and describe our progress towards compiling a λ-calculus with dependent types (Π-types and Σ-types) to our combinators.

1 Introduction

Simply-typed combinatory logics like the SK-combinators are known to be equivalent to the simply-typed λ-calculus [1]. These calculi can avoid contexts, and might enjoy benefits as such (e.g., unification). Work has been done on extending these logics with dependent types [2], but no implementation has been proposed, nor have such theories been formalized in a proof assistant.

Combinatory logics suffer from the known disadvantage that they are an inefficient compilation target. For example, when compiling the λ-calculus to the SK-combinators, terms explode in size due to lacking variable substitution [7, p. 267].

Numerous alternative combinatory compilation targets have been proposed. One such calculus is the categorical combinator calculus [4]. The categorical combinators are best described as a literal interpretation of cartesian closed categories as a Lisp-like Debruijn-indexed machine (the Categorical Abstract Machine [3]). Variable substitution is modeled intuitively as composing binary product projections [3, p. 6] over an environment of nested tuples. The categorical combinators have been extended with simple types [4]. There has also been some investigation into extending them to a dependent type theory similar to the Calculus of Constructions [8], but no implementation has been defined.

In this paper, we formalize a metatheory of a dependently-typed categorical combinator calculus with affine types and non-linear terms based on symmetric monoidal categories. We show that type inference is decidable and deterministic. We accomplish this goal in principle by observing that the types of the dependent non-linear categorical combinators are encodable with an affine subset of the usual reduction rules. By showing that the affine restriction of the calculus is strongly normalizing through a syntactic decreasing measure, we achieve total affine β-reduction. We show that the calculus satisfies global confluence by Newman's lemma [6] using our measure. We prove completeness and soundness of affine β-reduction, and soundness of β-normal forms with respect to our reduction relation, and hence achieve decidable affine equality. We are able to avoid a conversion rule in the typing judgment by eagerly normalizing types, and thus achieve deterministic, decidable type inference. We do not assume Π-extensionality, but describe η-reduction rules used for a weak form of extensionality as in Curien et al. [4, p. 221]. We outline several of our architectural decisions including proof automation with typeclasses critical to satisfying our formalization goals in Lean 4.

2 Background

2.1 The Untyped Categorical Combinators

The categorical combinators [4] are the basis of the Categorical Abstract Machine [3]. They are a potential compilation target for the λ-calculus [3]. Categorical combinators simulate variable substitution through projections over nested tuples ("couples", or binary products) constituting an "environment". Binders are simulated through a curry combinator (Λ) which applies its two arguments as a couple to the body ($\Lambda(x)$). The reduction rules for the calculus resemble the rules of the SK combinator calculus, and their categorical semantics are given by cartesian-closed categories [3, p. 178].

(ass)	$(x \circ y)z = x(yz)$
(fst)	$Fst(a, b) = a$
(snd)	$Snd(a, b) = b$
(dpair)	$\langle x, y\rangle z = (xz, yz)$
(curr)	$\Lambda(x)yz = x(y, z)$
(app)	$App(f, x) = fx$
(quote)	$`(x)y = x$

Figure 1: Reduction rules of the untyped combinators as seen in the Categorical Abstract Machine.

2.2 The Simply-Typed Categorical Combinators

Curien extends the combinators with type annotations [4, p. 220], enabling simple typing. Quotation is omitted in the simply-typed variant. The inference rules are as such:

$$\textsc{Id}\ \frac{\alpha : \text{Type}}{Id(\alpha) : \alpha \to \alpha} \qquad \textsc{Coup}\ \frac{x : \alpha, y : \beta}{(x, y) : \alpha \times \beta} \qquad \textsc{Comp}\ \frac{x : \beta \to \gamma, y : \alpha \to \beta}{(x \circ y) : \alpha \to \gamma}$$

$$\textsc{Fst}\ \frac{\alpha : \text{Type}, \beta : \text{Type}}{Fst(\alpha, \beta) : \alpha \times \beta \to \alpha} \qquad \textsc{Snd}\ \frac{\alpha : \text{Type}, \beta : \text{Type}}{Snd(\alpha, \beta) : \alpha \times \beta \to \beta} \qquad \textsc{Pair}\ \frac{x : \alpha \to \tau_1, y : \alpha \to \tau_2}{\langle x, y\rangle : \alpha \to \tau_1 \times \tau_2}$$

$$\textsc{Curr}\ \frac{x : (\alpha \times \beta) \to \tau}{\Lambda(x) : \alpha \to \beta \to \tau} \qquad \textsc{Appl}\ \frac{\alpha : \text{Type}, \beta : \text{Type}}{App(\alpha, \beta) : ((\alpha \to \beta) \times \alpha) \to \beta} \qquad \textsc{App}\ \frac{f : \alpha \to \beta x : \alpha}{fx : \beta}$$

Figure 2: Inference rules of the simply-typed categorical combinators

Curien's categorical combinators are nominally combinatory, but differ from SK and $BCKI$ calculi in that points are still present. In its simply-typed form [4, p. 220], the calculus has not one primitive combinator without a point, and no object (besides application) has an arity of more than one [3, p. 177]. Type arguments are given as points, projection mappings in pairs are given as points ($\langle f, g\rangle$), the Curry combinator is defined by a point ($\Lambda(x)$), couple members are specified as points ((a, b)), and composition is pointed with respect to f and g ($(f \circ g)x$). This superficially obscure design decision is quite natural when interpreting projection over couples ($((x_1, x_2), \ldots x_n)$) as debruijn argument indexing. Take an example reduction in the untyped calculus:

$$\begin{aligned}
&\Lambda(\Lambda(App \circ \langle Snd \circ Fst, App \circ \langle App \circ Fst, Snd\rangle\rangle))nfx \\
&= \Lambda(App \circ \langle Snd \circ Fst, App \circ \langle App \circ Fst, Snd\rangle\rangle)(n, f)x \\
&= (App \circ \langle Snd \circ Fst, App \circ \langle App \circ Fst, Snd\rangle\rangle)((n, f), x) \\
&= App(f, App(App(n, f), x)) \\
&= f(nfx)
\end{aligned} \tag{1}$$

Despite every visible operation being bound by a point, arguments in scope can be rearranged through projections, composition, pairing, and the application combinator which corresponds to evaluation in the cartesian closed category.

3 Dependent Categorical Combinators

Our primary insight when extending the categorical combinators to dependent types is that the types of all of the categorical combinators are representable without making use of the pair rule. That is, their types are affine. We refer to the subset of the categorical combinators excluding the pair rule as the "affine restriction." By showing that the affine restriction is strongly normalizing, we gain decidable judgmental equality ($\equiv$) and a decision procedure for type-checking.

We interpret Σ-types as a telescope context as usual. Finally, as in the Curry combinator, we specify combinator type arguments as term points that internalize an indexing of the environment (as in Tarski universes). For example, the Π combinator is inferred like such:

$$\frac{\delta : \text{Sort } m, f_\alpha : \alpha \to \text{Sort } n, f_\beta : \Pi(s : \delta)(x : f_\alpha s), \text{Sort } o}{\Pi(f_\alpha, f_\beta) : \delta \to \text{Sort } o}\ \text{PI} \tag{2}$$

We similarly regard couples as meaningful traversable data, but reinterpret them as Σ-pairs. Σ-couples make traversing data in a well-typed way quite convenient, and can naturally encode typing judgments. This approach also allows computing Π sort through our core inference rules without any modification. We define most of the type arguments of the combinators through Tarski universe environment-indexing points. By specifying type arguments in this way, some potential duplication of items in the context can be avoided (e.g., in Π, where the environment traverses both f_α and f_β), thus further enabling a purely affine type theory.

3.1 Couples as Degenerate Dependent Sums

Recall the usual typing of a Σ-pair:

$$\frac{\Gamma \vdash \alpha : \text{Sort } m, \beta : \alpha \to \text{Sort } n}{\Gamma \vdash \Sigma(\alpha, \beta) : \text{Sort } max(m, n)} \tag{3}$$

A Σ-type can be placed under a λ-binder, referencing bound variables in its fst and snd types, and fst itself in the type of snd. Σ-types are one of the canonical dependent types, alongside dependent products. There is a known equivalence between Π and Σ [5, p. 50], in that a function $\Pi(x : \alpha)(y : \beta(x)).\gamma(x, y)$ can be transformed into a dependent function $\Pi(s : \Sigma(\alpha, \beta)).\gamma(fst(s), snd(s))$, and vice-versa ("currying"-"uncurrying").

In the untyped categorical combinators, Curry-Uncurry can be written as:

$$\begin{aligned} \mathit{Curry}(x) &:= \Lambda(x) \\ \mathit{Uncurry}(x) &:= \mathit{App} \circ \langle x \circ Fst, Snd \rangle \\ \mathit{Uncurry}(x)(y, z) &= xyz \end{aligned} \tag{4}$$

In a dependently-typed categorical combinator calculus where couples are naively interpreted as Σ pairs with point-free type arguments, this (4) derivation becomes prohibitively difficult, especially in an affine setting. In $\mathit{Uncurry}(x)$, $snd\ (z : \beta y)$ depends on the fst component $(y : \alpha)$ of the encapsulating couple's preimage, despite y being destroyed upon reducing the pair application. App has no knowledge of y, and thus cannot calculate the type of (xy, z). We circumvent this contention by indexing Σ with a map of the environment $f_\beta : \Pi(s : \delta)(x : f_\alpha s).\text{Sort } n$ (5), similarly to the point x in Curry ($\Lambda(x)$).

We thus define couples $[\![\beta]\!](a, b)$ as a degenerate case of Σ, where the environment conveys no extra meaningful information, and β depends only on fst. We represent a no-op indexing of the environment with a unit type $\top$ so as to avoid universe arguments to Σ. For legibility, we use the notation $\mathit{Coup}(\beta)$ to refer to the type of a couple where $\beta : \alpha \to \text{Sort } n$ (**??**). We always refer to couples with their β

$$\frac{\textsc{Sigma}}{\alpha : \text{Sort } m, f_\beta : \delta \to \alpha \to \text{Sort } n} \over \Sigma(f_\beta) : \delta \to \text{Sort } max(m,n) \tag{5}$$

Figure 3: Where δ denotes an arbitrary environment and α denotes the fst component. $\Sigma(f_\beta)(env : \delta) : \text{Sort } max(m,n)$

$$\frac{}{\top : \text{Sort } 0}\ \textsc{Top} \qquad \frac{}{* : \top}\ \textsc{Unit} \qquad \frac{\textsc{Coup}}{A : \text{Sort } m, B : A \to \text{Sort } n, x : A, y : B(x),} \over [\![B]\!](x,y) : \Sigma(\text{`(Sort } m, B))A \qquad \frac{}{Coup(B) \coloneqq (\Sigma(\text{`}(\top, B))*)}$$

points, since they cannot be reasonable inferred. We use the notation $f(x,y)$ to denote application for readability purposes.

This view of couples as sigma terms naturally encodes the notion of a telescope context, and proves to be invaluable when compiling dependently-typed λ-calculi to the categorical combinators. For example, the term $[\![\text{Sort } m, Id(\text{Sort } m)]\!](\alpha, x)$ corresponds to the judgment $x : \alpha$. Similarly, a context Γ can be encoded as a nested couple:

$$x_0 : A_0, x_1 : A_1(x_0), x_2 : A_2(x_0, x_1) \approx [\![\Sigma(A_0, A_1), A_2]\!]([\![A_0, A_1]\!](x_0, x_1), x_2) \tag{6}$$

In the categorical abstract machine, this encoding of judgments corresponds to an environment where variable assignments are explicitly typed.

3.2 Type Annotations as Tarski Universes

All combinator objects have type annotations specified as points (usually as families indexing the environment). For example, the projection $Snd(f_\beta)$ is indexed by the term f_β which depends on an arbitrary environment $(s : \delta)$ and the term $fst : \alpha$. A projection for degenerate couples with no relevant environment can be derived, and we refer to it as $Sndc(\beta)$.

3.3 Inference Rules

We make use of an infinite sort hierarchy to avoid paradoxes (i.e., Sort m : Sort $m+1$). We use the notation $\Pi(x : A), B(x)$ to illustrate our inference rules. We omit some type points for legibility. However, the full combinatory types are accessible in the appendix (A). We use $A \to B$ as shorthand for our definition of arrows (8), but arrows are not first-class, and are merely an application of $\Pi(f_\alpha, f_\beta)s$.

3.3.1 Application and Π

The inference rule for function application is as usual, except the codomain β is applied the environment $s : \delta$, and the argument $x : f_\alpha s$. Since $f_\alpha s$ is discarded after type-checking, the environment can safely traverse the Π object (see inference rule (2)) without causing nontermination. In practice, we normalize types eagerly, but we do not always indicate so in our inference rules (see (A)). We use $\to^!$ to denote star-step affine reduction to a normal form.

$$\frac{\textsc{App}}{f : \Pi(f_\alpha, f_\beta)s, f_\alpha x \to^! \alpha \qquad x : \alpha_2, \alpha_2 \to^! \alpha \qquad f_\beta s x \to^! \beta} \over fx : \beta \tag{7}$$

Figure 4: Inference rule: application

Since we do not make use of variables, Π is itself a combinator $(\Pi(f_\alpha, f_\beta))$ dependent on an environment and the argument x. Since the dependency is internalized by f_α and f_β, its type can be expressed with an arrow (2).

3.3.2 Arrow types

We define nondependent arrows ($A \to B$) as such Π objects quoting their codomain to discard the environment-argument couple (8).

$$\frac{A : \text{Sort } m, B : \text{Sort } n}{A \to B \coloneqq (\Pi(Id(\text{Sort } m), \Lambda(\text{Sort } m, Id(\text{Sort } m), \text{`}(Coup(Id(\text{Sort } m)), B)))A)} \tag{8}$$

3.3.3 Couple Projections

We define dependent and nondependent versions of the Snd projection to bootstrap several of the combinators' types. We also derive a $Sndc$ combinator which accepts a coupe not depending on an environment. $Fst(\beta)$ is not dependent, since α does not refer to any object in context. Composition ($f \circ g$) is nondependent with respect to g, although f may depend on gx. A form of semi-dependent can be derived if g gives its dependency by a couple ($gx : \Sigma(\ldots)\ldots$). The inference rules for projection on Σ-terms are as such:

$$\textsc{Fst}\ \frac{\alpha : \text{Sort } m, \delta : \text{Sort } o, f_\beta : \delta \to \alpha \to \text{Sort } n}{Fst(\beta) : \Pi(env : \delta)(s : \Sigma(f_\beta)\ s), \alpha} \qquad \textsc{Snd}\ \frac{\alpha : \text{Sort } m, \delta : \text{Sort } o, f_\beta : \delta \to \alpha \to \text{Sort } n}{Snd(\beta) : \Pi(env : \delta)(s : \Sigma(f_\beta)\ s), f_\beta\ env\ (Fst(f_\beta)envs))}$$

$$\textsc{Snd'}\ \frac{\alpha : \text{Sort } m, \beta : \text{Sort } n}{Snd'(\alpha, \beta) : Coup(\text{`}(\alpha, \beta)) \to \beta} \qquad \frac{}{Sndc(\beta) \coloneqq Snd(\text{`}(\top, \beta))*} \qquad \frac{}{Fstc(\beta) \coloneqq Fst(\text{`}(\top, \beta))*}$$

3.3.4 Identity

Id accepts α as a point, but not a function of the environment. A type-point-free variant of Id can be derived easily with Curry (Λ). The type of $Id(\alpha)$ is a simple arrow:

$$\textsc{Id}\ \frac{\alpha : \text{Sort } m}{Id(\alpha) : \alpha \to \alpha} \tag{9}$$

3.3.5 Composition

We have nondependent composition as in Curien's combinators, except that the codomain of f can depend on gx, but the codomain of g cannot depend on x (10). Allowing both f and g to depend on their inputs would require duplicating the argument x, which would not satisfy our affine restriction on types.

$$\textsc{Nondep. comp.}\ \frac{f : \alpha \to \beta, g : \beta \to \gamma}{(g \circ f) : \alpha \to \gamma} \tag{10}$$

Figure 5: Inference rule: nondependent composition

Despite composition being only half-dependent, full dependency can be recovered when g produces a Σ-term encoding its dependency (i.e., $g : A \to Coup(B)$, where $B : A \to \text{Sort } n$).

3.3.6 Pairs

Recall $\langle f, g\rangle x = (fx, gx)$. We extend the pair operator with type and universe-level arguments m, α, f_β, and f_γ, which dictate the types of the resulting fst, and snd components, respectively. All arguments, up to x, are given by points (11). f and g may depend on x, but gx does not depend on fx, as this would require changing the pair rule.

$$\textsc{Pair}\quad \frac{f : \Pi(x : \alpha), \beta\alpha x \qquad g : \Pi(x : \alpha), \gamma\alpha x}{[\![m, \alpha, \beta, \gamma]\!]\langle f, g\rangle : \Pi(x : \alpha), \text{`(Sort } m, \Sigma(\Lambda(\alpha, \beta\alpha, \gamma \circ Fstc(f_\beta alpha))))} \tag{11}$$

3.3.7 Quotation

The quotation operator (`(α, β, x)) is not dependent, but a dependent version like the dependent K combinator can be derived.

$$\textsc{Quot}\quad \frac{\alpha : \text{Sort } m, \beta : \text{Sort } n, x : \alpha}{\text{`}(\alpha, \beta, x) : \beta \to \alpha} \tag{12}$$

3.3.8 Curry

Recall $\Lambda(x)yz = x(y, z)$. Since x is applied a Σ-couple, z must depend on y. That is, $z : \beta y$. However, x also depends on the couple $[\![\alpha, \beta]\!](y, z)$. Our inference rule for application (14) proves to be useful, since terms in context can be used more than once in a Π tree, so long as they are used at most once in each argument type.

$$\textsc{Curry}\quad \frac{\beta : \alpha \to \text{Sort } n, x : (\Pi(x : f_\alpha x_\alpha), \gamma x), (f_\alpha x_\alpha) \to^! Coup(\beta)}{\Lambda'(\alpha, \beta, x) : \Pi(x : \alpha)(y : \beta x), \Lambda(\alpha, \beta, \gamma x_\alpha)\ xy} \tag{13}$$

Figure 6: Inference rule: curry

3.3.9 App and Subst combinator

As in Curien's combinators, the *App* combinator evaluates a couple as a computation. We intuitively mirror the inference rule for Π, in that the types points for *App* embody Tarski universes (14). In fact, the types of the points f_α and f_β are the exact same as in Π. However, as in Π, f_β depends both on the environment, and f in the type of $App(f_\alpha, f_\beta)[\![\ldots]\!](f, x)$. Furthermore, since $f : \Pi(f_\alpha, f_\beta)(env : \delta)$, the environment must be duplicated across the codomain f_β and the *Snd* projection of $[\![\ldots]\!](f, x)$. This would seem to violate our affine requirement. Hence, we make use of a *Subst* construction, which behaves similarly to the S combinator. However, we view the environment as being computationally irrelevant. *Subst* can thus be viewed as performing context substitution, determining only the respective types. We only define *Subst* on projections, since no other usages are required to express our metatheory (although they may be beneficial for practical expressivity) (**??**).

$$\frac{}{Subst^{\to}_{Fst}\ Subst(x, Fst(f_\beta))env[\![\beta]\!](y, z) \to xenvy} \qquad \frac{}{Subst^{\to}_{Snd}\ Subst(x, Snd(f_\beta))env[\![\beta]\!](y, z) \to xenvz}$$

$$Subst_{Fst}\ \frac{x : \Pi(env : \delta)(x : \alpha), \gamma envx}{Subst(x, Fst(f_\beta)) : \Pi(env : \delta)(s : \Sigma(\beta)), Subst(\gamma, Fst(\beta))envs} \qquad Subst_{Snd}\ \frac{x : \Pi(env : \delta)(x : f_\beta env), \beta}{f_{\beta s} \coloneqq \Lambda(\delta, f_\alpha, f_\beta \circ Fstc(f_\alpha))Subst(x, Snd(f_{\beta s})) : \Pi(env :}$$

Using our *Subst* rules, we define the *App* inference rule (14).

Appl

$$\frac{\delta : \text{Sort } m, f_\alpha : \alpha \to \text{Sort } n, f_\beta : \Pi(s : \delta)(x : f_\alpha s), \text{Sort } o}{t_{\beta xy} \coloneqq \Lambda(\delta, \Pi(f_\alpha, f_\beta), f_\alpha \circ Fstc(\Pi(f_\alpha, f_\beta))), App(f_\alpha, f_\beta) : \Pi(env : \delta)(s : \Sigma(t_{\beta xy})env), Subst(f_\beta, Snd(t_{\beta xy}))} \quad (14)$$

Figure 7: *App* inference rule

4 Affine and η-Reduction

In general, dependent, point-free combinatory logics result in sizeable and complex types. We do not assume Π-extensionality, but make use of aggressive η-reduction rules based on Curien's rules for the simply-typed categorical combinators [4, p. 221]. We invoke β and η-reduction in our judgment relation. Hence, we prove soundness and completeness of reduction functions for each scheme. We do so by a syntactic decreasing measure. The derivations of completeness and soundness for β and η-reduction are quite similar structurally. Thus, we make the practical design decision of abstracting the notion of a size, step, and reduction relations with Lean's typeclass system.

4.1 Affine Single-Step Reduction

We formally define affine single-step reduction ($\rightarrowtail$) as such:

```
inductive RedAff : Expr → Expr → Prop
  | subst₁   : RedAff (DCC (Subst(x, Fst(f_β)) env ⟦β⟧(y, z))) (DCC (x env y))
  | subst₂   : RedAff (DCC (Subst(x, Snd(f_β)) env ⟦β⟧(y, z))) (DCC (x env z))
  | snd'     : RedAff (DCC (Snd'(_δ, _τ) ⟦β⟧(a, b))) b
  | quot     : RedAff (DCC (`(β, x) y)) x
  | id       : RedAff (DCC (Id(α) x)) x
  | appl     : RedAff (DCC (App(_α, _β) ⟦δ⟧(env, ⟦β⟧(x, y)))) (DCC (x y))
  | fst      : RedAff (DCC (Fst(_α) _env ⟦β⟧(a, b))) a
  | snd      : RedAff (DCC (Snd(_α) _env ⟦β⟧(a, b))) b
  | comp     : RedAff (DCC ((x ∘ y) z)) (DCC (x (y z)))
  | curr     : RedAff (DCC (Λ(α, β, x) y z)) (DCC (x ⟦β⟧(y, z)))
  | sigl     : RedAff x x' → RedAff (DCC ⟦β⟧(x, y)) (DCC ⟦β⟧(x', y))
  | sigr     : RedAff y y' → RedAff (DCC ⟦β⟧(x, y)) (DCC ⟦β⟧(x, y'))
  | congrFun : RedAff f f' → RedAff (DCC (f x)) (DCC (f' x))
  | congrArg : RedAff x x' → RedAff (DCC (f x)) (DCC (f x'))
```

Notably, none of our $\rightarrowtail$ rules duplicate any of their term arguments. Furthermore, each rule (except for congruence) eliminates at least one combinator. Hence, we define a syntactic notion of term size:

```
inductive Size : Expr → ℕ → Prop
  | tintro : Size (DCC ⋆) 0
  | top    : Size (DCC ⊤) 0
  | bot    : Size (DCC ⊥) 0
  | subst  : Size x m → Size y n → Size (DCC Subst(x, y)) (m + n).succ
  | sig    : Size (DCC Σ(β)) 0
  | coup   : Size x m → Size y n → Size (DCC ⟦β⟧(x, y)) (m + n)
  | sort   : Size (DCC Sort m) 0
  | pi     : Size (DCC Π(α, β)) 0
  | quot   : Size x m → Size (DCC `(β, x)) m.succ
  | id     : Size (DCC Id(α)) 1
  | fst    : Size (DCC Fst(_α)) 1
  | snd'   : Size (DCC Snd'(δ, β)) 1
  | snd    : Size (DCC Snd(_α)) 1
  | appl   : Size (DCC App(_α, β)) 1
  | comp   : Size f m → Size g n → Size (DCC (f ∘ g)) (m + n).succ
  | pair   : Size f m → Size g n → Size (DCC ⟦_m, α, β, γ⟧⟨f, g⟩) (m + n).succ
  | curr   : Size x m → Size (DCC Λ(α, β, x)) m.succ
  | app    : Size f m → Size x n → Size (DCC (f x)) (m + n)
```

Except for objects with no reduction rules, sizes are defined by at least the successor of the constituent computationally-relevant terms. For example, type points like α in Snd' are only of use to the kernel, so they have no weight. We account for permutation of term points in combinators (e.g., in composition) by incrementing their sizes to reflect the elimination of the combinator.

4.2 Relations

The structure of our completeness and soundness proofs for η, $\rightarrowtail$, and their respective closures, are quite similar. Hence, we identify a set of minimal requirements for completeness and strong normalization for our rules: soundness of a computable definition with respect to the inductive relation, a proof that our size measure decreases over the reduction, local confluence of only the respective single-step relations, and "semi-completeness". We define these properties as typeclasses and instantiate them for our η and $\rightarrowtail$ relations. By doing so, we automatically gain global confluence by Nemann's lemma, an instance of our reduction relation for the transitive closure, a total, terminating reduction function, strong normalization, well-foundedness with respect to the relation, uniqueness of normal forms, and, most importantly, completeness of the respective reduction functions and decidability of the reduction relation.

Our notion of a relation is defined by an inductive representation and a computable definition with soundness:

```
class Rel.{u, v, s} {α : Type u} {β : Type s} (r : α → β → Type v)
  (m : Type (max v s) → Type (max v s) := Id) [hM : Monad m] where
  run (x : α) : m ((y : β) × (r x y))
  sound       : run x = pure ⟨y, h⟩ → r x y := fun _ => h
```

We use universe and monadic polymorphism to enable expressing single-step reduction, size measures, star-step reduction, and multi-step reduction through `Rel`.

We define completeness with respect to a relation (`Rel`) as such:

```
open Rel in class Complete.{u, v, s} {α : Type u} {β : Type s} (r : α → β → Sort v)
  (P : β → Prop := fun _ => True)
  (m : Type (max v s) → Type (max v s) := Id) [hm : Monad m]
  [hR : Rel.{u, v, s} (fun (x : α) (y : β) => PULift.{v + 1, v} (r x y)) (m := m)] where
  complete : ∀ (h : r x y) (_p : P y), hR.run x = hm.pure ⟨y, PULift.up h⟩
```

We define a size measure, and are able to automatically derive completeness with tactics:

```
open Rel in class Measure (r : Expr → ℕ → Prop)
  extends Rel.{0, 0, 0} (fun x n => PULift.{1, 0} (r x n)), Complete.{0} r where
  complete := by
    intro x y h _h
    induction h
    all_goals (
      simp_all [pure, id_pure_ext]
      try rfl
      try grind
    )

abbrev Measure.size (t : outParam (Expr → ℕ → Prop)) [hM : Measure t]
  (e : Expr) : ℕ := (hM.run e).fst
```

We define a reduction relation as a relation satisfying the decreasing measure property. Similarly, we automatically derive the proof using tactics and custom aesop and simp-sets.

```
class RedRel (r : Expr → Expr → Prop) (t : outParam (Expr → ℕ → Prop))
  (m : Type → Type := Option) [_hM : Monad m]
  [hM : Measure t] extends Rel.{0, 0, 0} (fun (x : Expr)
  (y : Expr) => PULift.{1, 0} (r x y)) (m := m) where
  decreasing : r x y → (hM.run y).fst < (hM.run x).fst := by
    intro x y h
    induction h
    all_goals (
      try dsimp
      try simp_all [dcc_rel, dcc_size]
      try grind
      try aesop (rule_sets := [DccRel, DccSize])
      try rfl)
```

In doing so, we inherit strong normalization and well-foundedness:

```
def RedRel.sn {r : Expr → Expr → Prop} (t : outParam (Expr → ℕ → Prop))
  {m : Type → Type} [Monad m] [hM : Measure t]
  (hR : RedRel r t (m := m) (hM := hM)) (e : Expr) : Acc (fun b a => r a b) e :=
    Acc.intro e (fun e' hr => by
    have h := hR.decreasing hr
```

```
    exact sn t hR e')
termination_by (hM.run e).fst

def RedRel.wellFounded {r : Expr → Expr → Prop} (t : outParam (Expr → ℕ → Prop))
  {m : Type → Type} [Monad m] [hM : Measure t]
  (hR : RedRel r t (m := m) (hM := hM)) : WellFounded (fun a b => r b a) := ⟨hR.sn⟩
```

Using our decreasing measure proof, we derive a reduction function with soundness with respect to the reflexive transitive closure of a reduction relation:

```
def reduce {t : Expr → ℕ → Prop} [hT : Measure t] (r : Expr → Expr → Prop)
  [hR : RedRel r t (hM := hT)] (e : Expr) :
  ((e' : Expr) × (PULift.{1, 0} (Relation.ReflTransGen r e e'))) :=
  match hR.run e with
  | .some ⟨e', ⟨h⟩⟩ =>
    have _size_ : (hT.run e').fst < (hT.run e).fst := hR.decreasing h
    let ⟨e'', ⟨h₂⟩⟩ := reduce r (t := t) e'
    ⟨e'', ⟨.trans (.single h) h₂⟩⟩
  | .none => ⟨e, ⟨.refl⟩⟩
termination_by (hT.run e).fst

@[dcc_reduce, inline] abbrev reduce' {t : Expr → ℕ → Prop} [hT : Measure t] (r : Expr → Expr → Prop) [hR : Red
  (e : Expr) : Expr := (reduce r e).fst
```

We make extensive use of these derived reducers in our type inference implementation.

Since our $\rightarrowtail$ rules include congruence, we must prove confluence to demonstrate completeness of our β-reducer. We define local confluence like such:

```
open Rel in class Confluent.{u} {α : Type u} (r : α → α → Prop)
  (m : (α → α → Prop) → α → α → Prop := fun x => x) where
  conf : (m r A B) → (m r A C) → ∃ D, Relation.ReflTransGen.{u} r B D ∧ Relation.ReflTransGen.{u} r C D
```

By Newmann's lemma, we derive global confluence and uniqueness of normal forms given a decreasing measure under our definition of a reduction relation:

```
open Confluent in def RedRel.globally_confluent
  {r : Expr → Expr → Prop} {t : Expr → ℕ → Prop} {m : Type → Type}
  [hM : Measure t] [Monad m] [hC : Confluent r]
  [hR : RedRel r t (m := m) (hM := hM)] : (Confluent r (m := Relation.ReflTransGen)) where
  conf := fun {A B C} (h₁ : hR.ReflTrans A B) (h₂ : hR.ReflTrans A C) => by
    induction A using hR.wellFounded.induction generalizing B C with
    | h x ih =>
      match h₁.cases_head, h₂.cases_head with
      | .inl heq₁, .inl heq₂ =>
        exact ⟨x, heq₁ ▸ .refl, heq₂ ▸ .refl⟩
      | .inl heq₁, _ =>
        exact ⟨C, heq₁ ▸ h₂, .refl⟩
      | _, .inl heq₁ =>
        exact ⟨B, .refl, heq₁ ▸ h₁⟩
      | .inr ⟨C₁, hr₁, hrt₁⟩, .inr ⟨C₂, hr₂, hrt₂⟩ =>
        have ⟨D, hrt₁b, hrt₂b⟩ := hC.conf hr₁ hr₂
        have ⟨_, hrtt₁, hrtt₁b⟩ := ih _ hr₁ hrt₁ hrt₁b
        have ⟨_, hrtt₂, hrtt₂b⟩ := ih _ hr₂ hrt₂ hrt₂b
        have ⟨_, h₁, h₂⟩ := ih _ hr₁ (hrt₁b.trans hrtt₁b) (hrt₁b.trans hrtt₂b)
        exact ⟨_, .trans hrtt₁ h₁, .trans hrtt₂ h₂⟩

def IsNorm.unique_refl {r : Expr → Expr → Prop} (hn : IsNorm r A) (hr : (Relation.ReflTransGen r) A B) : A = B
  rw [← Relation.reflGen_transGen] at hr
  cases hr with
  | refl => rfl
  | single h => exact hn.unique_trans h

open Confluent in @[dcc_rel, aesop safe destruct (rule_sets := [DccRel])] theorem norm_unique {r : Expr → Expr
  [hC : Confluent r m] (h₁ : m r A B) (hn₁ : IsNorm r B) (h₂ : m r A C) (hn₂ : IsNorm r C) : B = C := by
  have ⟨D, hr₁, hr₂⟩ := hC.conf h₁ h₂
  have ha := hn₁.unique_refl hr₁
  have hb := hn₂.unique_refl hr₂
  simp_all

open Confluent in @[dcc_rel, aesop safe destruct (rule_sets := [DccRel])] theorem norm_unique {r : Expr → Expr
  [hC : Confluent r m] (h₁ : m r A B) (hn₁ : IsNorm r B) (h₂ : m r A C) (hn₂ : IsNorm r C) : B = C := by
  have ⟨D, hr₁, hr₂⟩ := hC.conf h₁ h₂
  have ha := hn₁.unique_refl hr₁
  have hb := hn₂.unique_refl hr₂
  simp_all
```

Finally, we require a weak form of completeness to derive full completeness with respect to normal-form reduction. That is, if the inductive reduction relation $A \to B$ can be constructed, then there must exist a proof h that some C is computable by *run*:

```
open Rel in class SemiComplete {t : Expr → ℕ → Prop} [hM : Measure t]
  (r : Expr → Expr → Prop)
  [hR : RedRel r t (m := Option) (hM := hM)] where
  complete : r A B → ∃ B h, (hR.run A) = .some ⟨B, ⟨h⟩⟩
```

Finally, we derive completeness of the reduction function with respect to our relation from confluence, an instance of `RedRel`, and semi-completeness:

```
theorem reduce_complete {r : Expr → Expr → Prop} {t : Expr → ℕ → Prop}
  [hM : Measure t] [hR : RedRel r t (m := Option.{0}) (hM := hM)] [hN : SemiComplete r (hR := hR)] [Confluent
  {A B : Expr} (hr : Relation.ReflTransGen r A B) (hn : IsNorm r B) : (reduce r A) = ⟨B, ⟨hr⟩⟩ := by
  simp [reduce]
  split
  case h_1 B' hr₂ heq =>
    have ⟨C, h₁, h₂⟩ := hR.globally_confluent.conf hr (.single hr₂)
    have h_b_c_eq := hn.unique_refl h₁
    have hr_b'_b := h_b_c_eq ▸ h₂
    match hr_b'_b.cases_head with
    | .inl h =>
      have h_red_norm := hR.reduce_norm_fst hn
      cases h_b_c_eq; cases h
      congr
      repeat simp_all
    | .inr ⟨_, hrB', hrt⟩ =>
      rename_i w
      have h_dec : (hM.size t B') < (hM.size t A) := by
        have h₁ := hR.decreasing hr₂
        have h₂ := hR.decreasing hrB'
        grind
      have h := reduce_complete (.trans (.single hrB') hrt) hn
      have h_fst : (reduce r B').fst = B := by
        simp_all
      cases h_fst
      congr
  case h_2 hneq =>
    match hr.cases_head with
    | .inl h => cases h; congr
    | .inr ⟨_, hrb', hrt⟩ =>
      have ⟨_, hA₁, hA₂⟩ := hN.complete hrb'
      simp_all
termination_by ((hM.size t A), (hM.size t B))
```

4.3 Instances: $\rightarrowtail$ reduction

Fortunately, most of our instances can be automated with our default tactics. We manually demonstrate local confluence and semi-completeness by case analysis (available in the appendix (A)). Finally, we inherit full completeness.

```
@[dcc_rel, aesop norm (rule_sets := [DccRel])] instance RedAff.instRedRel : RedRel RedAff Size where
  run := step?

@[dcc_rel, aesop norm (rule_sets := [DccRel])] instance RedAff.instCongruent : Congruent RedAff where

open Congruent in @[dcc_reduce, aesop safe apply (rule_sets := [DccReduce])] theorem RedAff.conf (h₁ : RedAff
  induction h₁ generalizing C with
  | subst₁ =>
    cases h₂ with
    | subst₁ => exact ⟨_, .refl, .refl⟩
    | congrFun h => cases h with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, congrFun' (congrArg' (.single h)), .single .subst₁⟩
    | congrArg h => cases h with
      | sigl h => exact ⟨_, congrArg' (.single h), .single .subst₁⟩
      | sigr h => exact ⟨_, .refl, .single .subst₁⟩
  ... omitted

def semi_complete {A B : Expr} (hr : A.RedAff B) : ∃ B', ∃ (h : A.RedAff B'), (step? A) = .some ⟨B', ⟨h⟩⟩ := b
  change ∃ _, ∃ _, (step? _) = .some _
```

```
  unfold step?
  split
  repeat exact ⟨_, (RedAff.instRedRel.sound rfl).down, rfl⟩
  case h_11 =>
    cases hr with
    | sigl h_x =>
      have ⟨_, hrx', heq'⟩ := semi_complete h_x
      exact ⟨_, .sigl (by assumption), by rw [heq']⟩
    | sigr h_y =>
      rename_i x y _
      have ⟨_, hry', heq'⟩ := semi_complete h_y
      match x.step? with
      | .none =>
        exact ⟨_, .sigr (by assumption), by rw [heq']⟩
      | .some ⟨_, ⟨h_x'⟩⟩ =>
        exact ⟨_, .sigl (by assumption), by rw [heq']⟩
  ... omitted

@[dcc_rel, aesop norm (rule_sets := [DccRel])] instance RedAff.instSemicomplete : SemiComplete RedAff := ⟨semi

@[dcc_rel, aesop norm (rule_sets := [DccRel])] instance instComplete : Complete (Relation.ReflTransGen RedAff)
```

We define our minimal notion of a relation by an inductive representation, and a computable implementation with soundness. We show that our size measure, single-step reduction, and normal-form reduction for the affine restriction and η, respectively, all satisfy our notion of a (computable) relation.

5 Reduction & Affine Reduction

Our reduction rules are largely the same as in the categorical abstract machine [3], with alterations and additions to enable affine dependent types (3.3).

$$
\begin{array}{ll}
\text{(id)} & Id(\alpha)x \to x \\
\text{(ass')} & (x \circ y)z \to x(yz) \\
\text{(ass)} & [\![\alpha, \beta, \gamma]\!] \circ xyz \to x(yz) \\
\text{(fst)} & Fst(\alpha, \beta)\ [\![\ldots]\!](a, b) \to a \\
\text{(snd)} & Snd(\alpha, \beta)\ [\![\ldots]\!](a, b) \to b \\
\text{(snd')} & Snd(\alpha, \beta)\ [\![\ldots]\!](a, b) \to b \\
\text{(curr')} & \Lambda'(\alpha, \beta, x)yz \to x\ [\![\alpha, \beta]\!](y, z) \\
\text{(curr)} & \Lambda(\alpha, x)yz \to x\ [\![\alpha, Id(\alpha)]\!](y, z) \\
\text{(app)} & App(n, \alpha, \beta)\ [\![\ldots]\!](f, x) \to fx \\
\text{(quote)} & `(\alpha, \beta, x)y \to x \\
\text{(flip)} & Flip(x)yz \to xzy \\
\text{(dpair)} & [\![\alpha, \beta]\!]\langle x, y\rangle\ z \to [\![\alpha, \beta]\!](xz, yz)
\end{array}
$$

$$
\text{(congr. fun.)}\quad \frac{f \to f'}{fx \to f'x} \qquad \text{(congr. arg.)}\quad \frac{x \to x'}{fx \to fx'}
$$

Figure 8: Reduction rules of the dependently-typed categorical combinators

5.1 Affine Strong Normalization

Note that, excluding our *dpair* rule, each single-step combinator reduction uses each of its arguments at most once. That is, this restriction of single-step reduction is affine. Except for congruence, each rule decreases the number of combinators (we do not consider the couple $[\![\alpha, \beta]\!](,)$ to be a combinator). We thus define a syntactic measure of term size on objects in our calculus, and show that it decreases under affine reduction ($\rightarrowtail$) to show that the affine reduction is strongly normalizing.

Formally, we define affine reduction ($\rightarrowtail$) as such:

```
inductive RedAff : Expr → Expr → Prop
  | app      : RedAff (DCC (App(_n, _δ, _τ) ⟦α, β⟧(f, x))) (DCC (f x))
  | fst      : RedAff (DCC (Fst(_δ, _τ) ⟦α, β⟧(a, b))) a
```

```
inductive Size : Expr → ℕ → Prop
  | sig    : Size (DCC Σ(_m, _n)) 0
  | coup   : Size (DCC ⟦α, β⟧(,)) 0
  | pi     : Size (DCC Π(_m, _n)) 0
  | sort   : Size (DCC Sort m) 0
  | id     : Size (DCC Id(α)) 1
  | comp   : Size (DCC ⟦α, β, γ⟧∘) 1
  | fst    : Size (DCC Fst(α, β)) 1
  | snd    : Size (DCC Snd(α, β)) 1
  | snd'   : Size (DCC Snd'(α, β)) 1
  | appl   : Size (DCC App(_n, α, β)) 1
  | flip   : Size e m → Size (DCC Flip(e)) m.succ
  | quot   : Size x m → Size (DCC `(α, β, x)) m.succ
  | comp'  : Size f m → Size g n → Size (DCC (f ∘' g)) (m + n).succ
  | pair   : Size f m → Size g n → Size (DCC ⟦α, β⟧⟨f, g⟩) (m + n).succ
  | curr   : Size x m → Size (DCC Λ(α, x)) m.succ
  | curr'  : Size x m → Size (DCC Λ'(α, β, x)) m.succ
  | app    : Size f m → Size x n → Size (DCC (f x)) (m + n)
```

Figure 9: Syntactic size measure for the combinators

```
  | snd      : RedAff (DCC (Snd(_δ, _τ) ⟦α, β⟧(a, b))) b
  | snd'     : RedAff (DCC (Snd'(_δ, _τ) ⟦α, β⟧(a, b))) b
  | flip     : RedAff (DCC (Flip(e) x y)) (DCC (e y x))
  | id       : RedAff (DCC (Id(_α) x)) x
  | quot     : RedAff (DCC (`(_α, _β, x) y)) x
  | comp     : RedAff (DCC ((⟦α, β, γ⟧∘ x y) z)) (DCC (x (y z)))
  | comp'    : RedAff (DCC ((x ∘' y) z)) (DCC (x (y z)))
  | curr     : RedAff (DCC (Λ(α, x) y z)) (DCC (x ⟦α, Id(α)⟧(y, z)))
  | curr'    : RedAff (DCC (Λ'(α, β, x) y z)) (DCC (x ⟦α, β⟧(y, z)))
  | congrFun : RedAff f f' → RedAff (DCC (f x)) (DCC (f' x))
  | congrArg : RedAff x x' → RedAff (DCC (f x)) (DCC (f x'))
```

We define the size of all reducible combinators to be the successor of the sum of the sizes of the combinators' constituent term points. For example, $Id(\alpha)$ has size 1, since its point is a term-type, not a term with computational relevance. Irreducible terms are assigned a size of 0 (9). We also define a computable `size (e : Expr) : ((n : ℕ) × (PLift (Size e n)))` function which produces a proof of `Size e n`, thus gaining soundness by default (definition omitted, see appendix).

Theorem 1. *If $A \in Expr$ and $A \rightarrowtail B$, then $size(B) < size(A)$.*

Proof.

```
theorem RedAff.decreasing (h : RedAff e e') : (size e').fst < (size e).fst := by
  induction h <;> (simp [size]; try grind)
```

□

We define strong normalization using Lean's accessibility predicate, and use our decreasing measure to produce a proof of `Acc (fun b a => RedAff a b) e`.

Theorem 2. *If $A \in Expr$, then A is strongly-normalizing under affine reduction ($\rightarrowtail$).*

Proof.

```
def sn (e : Expr) : Acc (fun b a => RedAff a b) e := Acc.intro e (fun e' hr => by
  have h := hr.decreasing
  exact e'.sn)
termination_by e.size.fst

instance instWellFounded : WellFounded (fun a b => RedAff b a) := ⟨sn⟩
```

□

5.2 Affine Global Confluence

We define the transitive ($\rightarrowtail^+$) and reflexive transitive closure ($\rightarrowtail^*$) like such:

```
def RedAffTrans     := Relation.TransGen RedAff
def RedAffReflTrans := Relation.ReflTransGen RedAff
```

Theorem 3. *For all $A \in Expr$, A is confluent under $\rightarrowtail$.*

Proof. We proceed by induction and case analysis on $\rightarrowtail$. Cases have been omitted for brevity. Consult the appendix.

```
theorem RedAff.conf (h₁ : RedAff A B) (h₂ : RedAff A C) :
  ∃ D, RedAffReflTrans B D ∧ RedAffReflTrans C D := by
  induction h₁ generalizing C with
  | comp =>
    cases h₂ with
    | comp => exact ⟨_, .refl, .refl⟩
    | congrFun h =>
      cases h with
      | congrFun h =>
        cases h with
        | congrFun h => cases h
        | congrArg h => exact ⟨_, .single (.congrFun h), .single .comp⟩
      | congrArg h => exact ⟨_, .single (.congrArg (.congrFun h)), .single .comp⟩
      | congrArg h =>
        exact ⟨_, .single (.congrArg (.congrArg (by assumption))), .single .comp⟩
  | congrFun h_r ih =>
    cases h₂
    case comp' | quot | id | app | fst | snd | snd' => cases h_r
    case flip =>
      cases h_r with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, .single .flip, .single (.congrArg h)⟩
    case comp =>
      cases h_r with
      | congrFun h =>
        cases h with
        | congrFun h => cases h
        | congrArg h => exact ⟨_, .single .comp, .congrFun (.single h)⟩
      | congrArg h => exact ⟨_, .single .comp, .congrArg (.congrFun (.single h))⟩
    case curr =>
      cases h_r with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, .single .curr, .single (.congrArg (.sigl (by assumption)))⟩
    case curr' =>
      cases h_r with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, .single .curr', .single (.congrArg (.sigl (by assumption)))⟩
    case congrFun f f' x f'' h =>
      have ⟨_, h₁, h₂⟩ := ih h
      exact ⟨_, .congrFun h₁, .congrFun h₂⟩
    case congrArg f f' x x'' h_x =>
      exact ⟨_, .congrArg (.single h_x), .congrFun (.single h_r)⟩
  | _ =>
    -- remaining cases omitted
    sorry
```

□

Theorem 4. *For all $A \in Expr$, A is confluent under $\rightarrowtail^*$.*

Proof. We proceed by induction on the well-founded relation $\rightarrowtail$ (5.1).

```
theorem RedAffReflTrans.conf (h₁ : RedAffReflTrans A B) (h₂ : RedAffReflTrans A C) :
  ∃ D, RedAffReflTrans B D ∧ RedAffReflTrans C D := by
  induction A using instWellFounded.induction generalizing B C with
  | h _ ih =>
    match h₁.cases_head, h₂.cases_head with
    | .inr ⟨_, hra, hrc⟩, .inr ⟨_, hra', hrc'⟩ =>
      have ⟨_, c₁', c₂'⟩ := hra.conf hra'
      have ⟨_, hrB, hrB'⟩ := ih _ hra hrc c₁'
      have ⟨_, hrC, hrC'⟩ := ih _ hra' hrc' c₂'
      have ⟨_, h₁, h₂⟩ := ih _ hra (c₁'.trans hrB') (c₁'.trans hrC')
      exact ⟨_, .trans hrB h₁, .trans hrC h₂⟩
    | .inl heq₁, .inl heq₂ =>
      rw [← heq₁, ← heq₂]
      exact ⟨_, .refl, .refl⟩
    | .inl heq₁, .inr _ =>
      rw [← heq₁]
      exact ⟨_, h₂, .refl⟩
    | .inr _, .inl heq₁ =>
```

```
    rw [← heq₁]
    exact ⟨_, .refl, h₁⟩
```

□

Theorem 5. *For all $A \in Expr$, if $A \rightarrowtail B$ and $A \rightarrowtail C$, and B and C are normal forms, then $B = C$.*

Proof. We define normal forms as such:

```
def IsNormAff (e : Expr) : Prop := ¬ ∃ e', RedAff e e'
```

We proceed by induction on the common reduct of B and C:

```
theorem RedAff.norm_unique (h₁ : RedAff A B) (h_norm : B.IsNormAff)
  (h₂ : RedAff A C) (h_norm₂ : C.IsNormAff) : B = C := by
  have ⟨D, ha, hb⟩ := h₁.conf h₂
  match ha.cases_head, hb.cases_head with
  | .inl h₁, .inl h₂ =>
    simp_all
  | .inr ⟨c₁, h₁, h₂⟩, _ =>
    exact h_norm.elim ⟨_, h₁⟩
  | _, .inr ⟨c₁, h₁, h₂⟩ =>
    exact h_norm₂.elim ⟨_, h₁⟩
```

□

Theorem 6. *For all $A \in Expr$, if $A \rightarrowtail^* B$ and $A \rightarrowtail^* C$, and B and C are normal forms, then $B = C$.*

Proof. We proceed by induction on the common reduct of B and C.

```
theorem RedAffReflTrans.norm_unique (h₁ : RedAffReflTrans A B) (h_norm : B.IsNormAff)
  (h₂ : RedAffReflTrans A C) (h_norm₂ : C.IsNormAff) : B = C := by
  have ⟨D, ha, hb⟩ := h₁.conf h₂
  match ha.cases_head, hb.cases_head with
  | .inl h₁, .inl h₂ =>
    simp_all
  | .inr ⟨c₁, h₁, h₂⟩, _ =>
    exact h_norm.elim ⟨_, h₁⟩
  | _, .inr ⟨c₁, h₁, h₂⟩ =>
    exact h_norm₂.elim ⟨_, h₁⟩
```

□

5.3 Decidable Beta-Equality

We give a decision procedure for $=_\beta$. We do so by proving soundness and completeness for β-reduction (with respect to normal-forms). We thus implement an instance of:

```
∃ C, RedAffReflTrans A C ∧ RedAffReflTrans B C ∧ C.IsNormAff
```

First, we define `step` and `reduce` functions with soundness using our measure:

```
def step? (e : Expr) : Option ((e' : Expr) × (PLift (RedAff e e'))) :=
  match e with
  | (DCC (App(_n, _δ, _τ) ⟦_α, _β⟧(f, x)))   => .some ⟨(DCC (f x)), ⟨.app⟩⟩
  | (DCC (Fst(_δ, _τ) ⟦_α, _β⟧(a, _b)))      => .some ⟨a, ⟨.fst⟩⟩
  | (DCC (Snd(_δ, _τ) ⟦_α, _β⟧(_a, b)))      => .some ⟨b, ⟨.snd⟩⟩
  | (DCC (Snd'(_δ, _τ) ⟦_α, _β⟧(_a, b)))     => .some ⟨b, ⟨.snd'⟩⟩
  | (DCC (Flip(e) x y))                      => .some ⟨(DCC (e y x)), ⟨.flip⟩⟩
  | (DCC (Id(_α) x))                         => .some ⟨x, ⟨.id⟩⟩
  | (DCC (`(_α, _β, x) _y))                  => .some ⟨x, ⟨.quot⟩⟩
  | (DCC ((⟦_α, _β, _y⟧∘ x y) z))            => .some ⟨(DCC (x (y z))), ⟨.comp⟩⟩
  | (DCC ((x ∘' y) z))                       => .some ⟨(DCC (x (y z))), ⟨.comp'⟩⟩
  | (DCC (Λ(α, x) y z))                      => .some ⟨(DCC (x ⟦α, Id(α)⟧(y, z))), ⟨.curr⟩⟩
  | (DCC (Λ'(α, β, x) y z))                  => .some ⟨(DCC (x ⟦α, β⟧(y, z))), ⟨.curr'⟩⟩
  | DCC (f x) =>
    match f.step?, x.step? with
    | .some ⟨f', ⟨h₁⟩⟩, _ => .some ⟨(DCC (f' x)), ⟨.congrFun h₁⟩⟩
    | _, .some ⟨x', ⟨h₁⟩⟩ => .some ⟨(DCC (f x')), ⟨.congrArg h₁⟩⟩
    | _, _ => .none
  | _ => .none
```

```
def reduce (e : Expr) : Expr :=
  match step? e with
  | .some ⟨e', ⟨h⟩⟩ =>
    have _h_size : e'.size.fst < e.size.fst := h.decreasing
    e'.reduce
  | .none => e
```

Theorem 7. *For all $A \in Expr$, and* `reduce A = B`*, then $A \rightarrowtail^* B$.*

Proof.

```
theorem reduce_sound (heq : reduce A = B) : RedAffReflTrans A B := by
  induction A using instWellFounded.induction generalizing B with
  | h a ih =>
    unfold reduce at heq
    split at heq
    case h_1 h_red h_step_eq =>
      exact Relation.ReflTransGen.trans (.single h_red) (ih _ h_red heq)
    case h_2 => simp_all only [forall_eq']; exact .refl
```

□

By using our decreasing measure and showing completeness for `step?` with respect to normal forms under $\rightarrowtail$, we prove completeness of `reduce` with respect to normal forms under $\rightarrowtail^*$.

Lemma 1. *For all $e \in Expr$, if e is a normal form, then* `step? e = none`*.*

Proof.

```
  theorem IsNormAff.complete (h : A.IsNormAff) : (step? A) = .none := by
  apply Option.eq_none_iff_forall_ne_some.mpr
  intro ⟨e', ⟨h₂⟩⟩ heq
  exact (h ⟨_, h₂⟩)
```

□

Theorem 8. *For all $A \in Expr$, if $A \rightarrowtail B$ and B is a normal form, then* `step A = some B`*.*

Proof. We proceed by case analysis with respect to the unfolding of `step?`:

```
theorem RedAff.complete (h₁ : RedAff A B) (h_norm : B.IsNormAff) :
  (step? A) = .some ⟨B, ⟨h₁⟩⟩ := by
  unfold step?
  split
  case h_4 | h_3 | h_2 | h_1  =>
    cases heq : h₁
    · rfl
    · rename_i h_r
      have h := h_r.complete h_norm.congrFun
      simp [step?] at h
    · rename_i h_r
      cases h_r
      · rename_i h_r
        cases h_r
        · rename_i h; cases h
        · have h := h_norm.complete
          simp [step?] at h
      · have h := h_norm.complete
        simp [step?] at h
  case h_7 | h_6 =>
    cases heq : h₁
    · rfl
    · rename_i h; cases h
    · have h := h_norm.complete
    simp [step?] at h
  case h_5 | h_8 | h_9 | h_10 | h_11 | h_12 | h_13 =>
    sorry -- omitted for brevity. See appendix.
```

□

Lemma 2. *For all $e \in Expr$, if e is a normal form, then* `reduce e = e`*.*

Proof.

```
theorem IsNormAff.complete_reduce (h : A.IsNormAff) : reduce A = A := by
  unfold reduce
  rw [h.complete]
```

□

Theorem 9. *For all $A \in Expr$, if $A \rightarrowtail^* B$, and B is a normal form, then* `reduce A = B`.

Proof. By well-founded recursion with respect to the decreasing measure `size`:

```
theorem RedAffReflTrans.complete (h₁ : RedAffReflTrans A B) (h_norm : B.IsNormAff) :
  reduce A = B := by
  match h₁.cases_head with
  | .inl h =>
    simp_all
    exact h_norm.complete_reduce
  | .inr ⟨c, h₁, h₂⟩ =>
    have _h : sizeOf c < sizeOf A := h₁.decreasing
    have h := RedAffReflTrans.complete h₂ h_norm
    rw [h₁.reduce_same, h]
termination_by A.size.fst
```

□

Finally, we show $=_\beta$ is decidable.

Theorem 10. *For all $A \in Expr$ and $B \in Expr$, $(\exists C, A \rightarrowtail^* C \wedge B \rightarrowtail^* C \wedge IsNorm(C)) \vee \neg(\exists C, A \rightarrowtail^* C \wedge B \rightarrowtail^* C \wedge IsNorm(C))$.*

Proof. This follows from completeness and soundness of `reduce`:

```
instance : Decidable (∃ C, RedAffReflTrans A C ∧ RedAffReflTrans B C ∧ C.IsNormAff) :=
  if heq : reduce A = reduce B then .isTrue
    ⟨_, reduce_sound rfl, reduce_sound heq.symm, reduce_norm rfl⟩
  else
    .isFalse (fun ⟨C, h₁, h₂, h_norm⟩ => by
      have ha := h₁.complete h_norm
      have hb := h₂.complete h_norm
      simp_all)
```

□

6 Type Inference

Note that the types of all of the dependent categorical combinators are affine (3.3). Hence, we use our decision procedure (5.3) for $=_\beta$ under affine-reduction to prove decidability of type-checking.

We do not normalize inside our judgment relation, nor do we have a conversion case. We bake conversion into each applicable constructor with a `Reduces` predicate by normalizing the expected and actual types a la carte:

```
inductive Reduces : Expr → Expr → Type
  | mk : RedAffReflTrans A B → IsNormAff B → Reduces A B
deriving Repr

theorem Reduces.complete : Reduces A B → reduce A = B :=
  fun ⟨ha, hb⟩ => ha.complete hb

def reduce_reduces (h : reduce A = B) : Reduces A B := ⟨reduce_sound h, reduce_norm h⟩

def reduce_reduces' : Reduces A A.reduce := reduce_reduces rfl
```

For example, application is inferred like such:

```
inductive Judge : Expr → Expr → Type where
  | app   : Judge f t_f
    → Reduces t_f (DCC (Π(m, n) α β))
    → Judge x t_x
    → Reduces t_x α
    → Judge (DCC (f x)) (DCC (β ⟦Sort m, Id(Sort m)⟧(α, x)))
```

We defer full type normalization to a supplementary predicate, `JudgeConv`:

```
inductive JudgeConv : Expr → Expr → Type
  | mk : Judge x A → Reduces A B → JudgeConv x B
```

We define an `IsArrow` predicate for convenience in pattern matching arrows, which are by convention, Π-objects containing quotation in their codomain. We do not assume Π-extensionality.

```
inductive IsArrow : Expr → (α β : Expr) → Level → Level → Type
  | mk    : Reduces A (arrow α β m n) → IsArrow A α β m n
deriving Repr

def is_arrow (t : Expr) : Option ((α β : Expr) × (m n : Level) × IsArrow t α β m n) :=
  match heq₀ : t.reduce with
  | DCC (Π(m, n) α `(Sort n₂, Σ(Sort m₂, Id(Sort m₃)), β)) =>
    if heq : m = m₂ ∧ m = m₃ ∧ n = n₂ then by
      have ⟨ha, hb, hc⟩ := heq
      rw [← ha, ← hb, ← hc] at heq₀
      refine .some ⟨α, β, m, n, ?_, ?_⟩
      · exact reduce_sound heq₀
      · exact reduce_norm heq₀
    else .none
  | _ => .none
```

Our `Judge` relation models the inference rules (3.3). We have omitted the individual types and corresponding constructors for brevity, but give the constructor for tail-dependent Curry (**??**) and App (14) as examples. See the appendix and supplementary materials for the full formalization.

```
def curr.type (α γ : Expr) (m n : Level) : Expr :=
  (DCC (Π(m.succ, n) α ((Flip(Π(m, n)) (γ ∘'
  (Λ'(Sort m.succ, Id(Sort m.succ), Id(Σ(Sort m.succ, Id(Sort m.succ))))
    ⟦Sort m.succ, Id(Sort m.succ)⟧(α, Id(α))))) ∘' Snd'(Sort m.succ, α))))

def app_e.type (α β : Expr) (m n : Level) : Expr :=
  let t_f := DCC (Π(m, n) α β)
  let t_x := DCC `(Sort n, t_f, α)

  DCC (Π(max m n, n)
      Σ(t_f, t_x)
      ((Λ'(Sort m, Id(Sort m), β) α) ∘' (Snd(t_f, t_x) ∘ Snd'(Sort (max m n), Σ(t_f, t_x)))))

inductive Judge : Expr → Expr → Type where
  | curr  : Judge α t_α
    → Reduces t_α (DCC (Sort m.succ))
    → Judge x t_x
    → Reduces t_x (DCC (Π(m.succ, n) Σ(α, Id(α)) γ))
    → Judge (DCC Λ(α, x)) (curr.type α γ m n)
  | app_e : Judge α t_α
    → Reduces t_α (DCC Sort m)
    → Judge (DCC App(n, α, β)) (app_e.type α β m n)
  | app   : Judge f t_f
    → Reduces t_f (DCC (Π(m, n) α β))
    → Judge x t_x
    → Reduces t_x α
    → Judge (DCC (f x)) (DCC (β ⟦Sort m, Id(Sort m)⟧(α, x)))
  | sort  : Judge (DCC Sort m) (DCC Sort m.succ)
deriving Repr
```

6.1 Computing Type Inference

Using our terminating affine β-reduction function and `reduce_reduces`, we define an `infer` function with soundness. We give only the cases described in our `Judge` illustration, though the rest are available in supplementary materials. Pattern matching against our `Judge` constructors is somewhat obtuse, primarily due to our design decision to avoid adding a conversion constructor and reducing the inferred types.

```
def infer (e : Expr) : Option ((t : Expr) × (Judge e t)) :=
  match e with
  | DCC Sort m => .some ⟨DCC Sort m.succ, .sort⟩
  | DCC App(n, α, β) =>
    match α.infer with
    | .some ⟨t_α, h_t_α⟩ =>
```

```
      match heq₀ : t_α.reduce with
      | DCC Sort m =>
        .some ⟨_, .app_e h_t_α (reduce_reduces heq₀)⟩
      | _ => .none
    | _ => .none
  | DCC Λ(α, x) =>
    match heq₀ : α.infer, x.infer with
    | .some ⟨t_α, h_t_α⟩, .some ⟨t_x, h_t_x⟩ =>
      match heq₁ : t_α.reduce with
      | DCC Sort m + 1 =>
        match heq₃ : t_x.reduce with
        | (DCC (Π(m₂ + 1, n) Σ(α₁, Id(α₂)) Y₂)) =>
          if heq₂ : m₂ = m ∧ α₁ = α ∧ α = α₂ then
            have ⟨ha, hb, hc⟩ := heq₂
            .some ⟨_, Judge.curr
              h_t_α
              (reduce_reduces heq₁)
              h_t_x
              (ha ▸ hb ▸ hc ▸ (reduce_reduces heq₃))⟩
          else .none
        | _ => .none
      | _ => .none
    | _, _ => .none
  | DCC (f x) => match f.infer, x.infer with
    | .some ⟨t_f, h_f⟩, .some ⟨t_x, h_x⟩ =>
      match heq₀ : t_f.reduce, heq₁ : t_x.reduce with
      | DCC (Π(m, n) α₁ β), α₂ =>
        if heq : α₁ = α₂ then
          .some ⟨(DCC (β ⟦Sort m, Id(Sort m)⟧(α₁, x))),
            .app h_f (reduce_reduces heq₀)
                h_x (reduce_reduces (heq ▸ heq₁))⟩
        else .none
      | _, _ => .none
    | _, _ => .none
  | _ => sorry -- remaining cases omitted for brevity

def infer_conv (e : Expr) : Option ((t : Expr) × JudgeConv e t) :=
  match infer e with
  | .some ⟨t, h⟩ =>
    .some ⟨t.reduce, ⟨h, reduce_reduces rfl⟩⟩
  | _ => .none
```

Avoiding a conversion constructor proves to be quite advantageous, since neither confluence or induction are necessary, and case analysis suffices to prove useful properties. For example, a proof of preservation of $Id(\alpha)x$ is quite straightforward.

Theorem 11. *For all $\alpha \in Expr$ and $x \in Expr$, if $Id(\alpha)\ x : reduce(\alpha)$, then $x : reduce(\alpha)$.*

Proof. We proceed by case analysis on the judgment $Id(\alpha)\ x : reduce(\alpha)$.

```
def id_preservation (h : JudgeConv (DCC (Id(α) x)) α.reduce) : (JudgeConv x α.reduce) :=
  match h with
  | ⟨.app (.id _ _) b h_t_x _, h_red⟩ => by
    cases reduce_arrow b.complete
    exact ⟨h_t_x, by assumption⟩
```

□

6.1.1 Infer Completeness

Yet another advantage of avoiding a conversion constructor, and of having decidable $\beta_=$ is that completeness is provable. We have omitted all but the `curr`, `app_e`, `sort`, and `app` constructors (see supplementary materials).

Theorem 12. *For all $e \in Expr$ and $t \in Expr$, if $e : t$, then `infer e = some t`.*

Proof. We proceed by parallel induction on the typing derivation and case analysis on the unfolding of `infer`.

```
theorem Judge.complete (h_t : Judge e t) : infer e = .some ⟨t, h_t⟩ := by
  induction h_t with
  | sort =>
```

```
    unfold infer
    rfl
  | curr h_t_α h_red₁ h_t_x h_red₂ ih₁ ih₂ =>
    have ha := h_red₁.complete
    have hb := h_red₂.complete
    unfold infer
    rw [ih₁, ih₂]
    simp
    split
    case h_1 heq =>
      rw [heq] at ha
      cases ha
      split
      case h_1 heq =>
        rw [heq] at hb
        cases hb
        simp
        split
        rfl
      case h_2 h =>
        exact (h _ _ _ _ _ hb).elim
    case h_2 h =>
      exact (h _ ha).elim
  | app_e h_t_α h_red₁ ih =>
    have ha := h_red₁.complete
    unfold infer
    rw [ih]
    simp
    split
    case h_1 heq =>
      rw [heq] at ha
      cases ha
      rfl
    case h_2 h =>
      exact (h _ ha).elim
  | app h_t_f h_red_f h_t_x h_red_x ih₁ ih₂ =>
    have ha := h_red_f.complete
    have hb := h_red_x.complete
    unfold infer
    simp [ih₁, ih₂]
    split
    case h_1 heq =>
      rw [heq] at ha
      cases ha
      cases hb
      simp
      aesop
    case h_2 h =>
      exact (h _ _ _ _ ha).elim
```

□

6.2 Judgmental Decidability

Using our proof of completeness and soundness proof produced by `infer`, we demonstrate judgmental decidability.

Theorem 13. *For all $e \in Expr$ and $t \in Expr$, $e : t \vee \neg e : t$.*

Proof.

```
def Judge.decide : Judge e t ⊕ (Judge e t → PLift False) :=
  if h : (infer e).isSome then
    if heq : ((infer e).get h).fst = t then
      .inl (heq ▸ ((infer e).get h).snd)
    else
      .inr fun hn => by
        have h_comp := hn.complete
        simp_all
  else .inr fun hn => by simp [hn.complete] at *

def JudgeConv.decide : JudgeConv e t ⊕ (JudgeConv e t → PLift False) :=
  if h : (infer e).isSome then
    if heq : ((infer e).get h).fst.reduce = t then
```

```
    .inl ⟨((infer e).get h).snd, reduce_reduces heq⟩
  else .inr fun ⟨ha, hb⟩ => by
    have h := ha.complete
    simp_all
    exact (heq hb.complete).elim
else .inr fun ⟨ha, hb⟩ => by simp [ha.complete] at *
```

□

7 Future Work

We hope to use our inference and reduction machinery to mechanize a proof of strong normalization of the entire calculus in the future. That is, a proof of strong normalization for the calculus including the pair combinator, which is not affine.

8 Conclusions

Our formalization gives a novel extension of the simply-typed categorical combinators [4, p. 220] to a dependently-typed setting. By mechanizing the theory in Lean 4, we provide several indispensable guarantees about the logic. Most importantly, we demonstrate a mechanism for deciding type inference of the calculus. We demonstrate strong normalization of an affine restriction on the categorical combinators, and use this property to prove soundness and completeness of an inference function with respect to typing judgments. In doing so, we also demonstrate a useful technique to avoid conversion in the typing relation, given we have decidable $\beta_{\to *}$.

Our proof of judgmental decidability is made possible in principle by the observation that the types of the dependent categorical combinators are purely affine. This fact is not true of equivalent systems, like the dependent SK combinators.

Finally, since the categorical combinators are well-known to be a performant compilation target, we believe that our verified implementation could have ramifications towards the efficient execution of dependent type inference in a purely contextless way.

Acknowledgements

To Jake, for teaching me lambda-calculus.

A Supplementary Materials

Lean 4 formalization repository: https://github.com/lexzaiello/dcc
Omitted inference cases: https://github.com/lexzaiello/DCC/blob/master/Dcc/CC/Judge.lean
Omitted reduction cases: https://github.com/lexzaiello/DCC/blob/master/Dcc/CC/Step2.lean

B Todos left

- Notation inconsistencies: I mix and match $f(x, y)$ and fxy. fxy is the preferred notation. Mixing and matching notation for our calculus and normal inference rule notation is also ambiguous and confusing. How to handle this?
- My notation for application is ugly. Usually application is written as fx, not $f\ x$
- More background: want a reference for telescope
- More background: want a reference for Tarski universes that isn't ncatlab
- If time permits, I want to finish the λ-calculus compiler.

References

[1] Thorsten Altenkirch et al. "Combinatory Logic and Lambda Calculus Are Equal, Algebraically". In: *International Conference on Formal Structures for Computation and Deduction*. 2023. URL: https://api.semanticscholar.org/CorpusID:259298739.

[2] Thorsten Altenkirch et al. "The Münchhausen Method in Type Theory". In: *28th International Conference on Types for Proofs and Programs (TYPES 2022)*. Ed. by Delia Kesner and Pierre-Marie Pédrot. Vol. 269. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 10:1–10:20. ISBN: 978-3-95977-285-3. DOI: 10.4230/LIPIcs.TYPES.2022.10. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2022.10.

[3] G. Cousineau, P-L. Curien, and M. Mauny. "The categorical abstract machine". In: *Functional Programming Languages and Computer Architecture*. Ed. by Jean-Pierre Jouannaud. Berlin, Heidelberg: Springer Berlin Heidelberg, 1985, pp. 50–64. ISBN: 978-3-540-39677-2.

[4] P.-L. Curien. "Categorical combinators". In: *Information and Control* 69.1 (1986), pp. 188–254. ISSN: 0019-9958. DOI: https://doi.org/10.1016/S0019-9958(86)80047-X. URL: https://www.sciencedirect.com/science/article/pii/S001999588680047X.

[5] *Intuitionistic type theory*. Vol. 9.

[6] Maxwell Herman Alexander Newman. "On theories with a combinatorial definition of" equivalence"". In: *Annals of mathematics* 43.2 (1942), pp. 223–243.

[7] Simon L. Peyton Jones. *The Implementation of Functional Programming Languages (Prentice-Hall International Series in Computer Science)*. USA: Prentice-Hall, Inc., 1987. ISBN: 013453333X.

[8] Eike Ritter. *Categorical combinators for the calculus of constructions*. Tech. rep. University of Cambridge, Computer Laboratory, 1990.